Workflow สำหรับการวิเคราะห์ผล nanopore ของห้องปฏิบัติการไวรัสวิทยา ศวพ.พิษณุโลก

คู่มือ Workflow ฉบับสมบูรณ์และสมบูรณ์แบบที่สุด ตั้งแต่เริ่มต้นไฟล์ผ่านโปรแกรม Minknow จากการทำ basecalling และการ trim ส่วน barcode มาแล้วได้ไฟล์ .fastq.gz จำนวนมากจากเครื่อง Nanopore มีความยาวต่อเนื่องไปจนถึงขั้นตอนสุดท้าย พร้อมคำสั่ง Command Line และสคริปต์ Python สำหรับตรวจสอบคุณภาพและสร้างกราฟในแต่ละสเต็ป

การรันการวิเคราะห์โปรแกรม bioinformatics บนระบบปฏิบัติการ Microsoft Windows สามารถทำได้สะดวกและแนะนำให้ทำผ่าน WSL2 (Windows Subsystem for Linux) เนื่องจากเครื่องมือ bioinformatics ส่วนใหญ่ (รวมถึง medaka, minimap2 และ samtools) ถูกออกแบบมาให้ทำงานบนระบบ Linux เป็นหลัก ขั้นตอนทั้งหมดตั้งแต่เปิดใช้งาน WSL2 ติดตั้งโปรแกรม จนถึงสิ้นสุดการวิเคราะห์ มีดังนี้

การวิเคราะห์ผลใน SOP นี้ทำการวิเคราะห์ด้วย path ดังนี้ C:/Chim_nanopore/barcode21/

การใช้งาน python ให้เข้าใช้งานผ่าน lib ที่ติดตั้งใน Ubuntu

ขั้นตอนที่ 1 : ติดตั้ง WSL2 และ Ubuntu บน Windows โดยเปิด PowerShell หรือ Command Prompt ด้วยสิทธิ์ Administrator (คลิกขวา -> Run as administrator) รันคำสั่งติดตั้งในส่วน WSL ใน PowerShell คือ

wsl –install

ทำการรีสตาร์ตเครื่อง Windows 1 รอบเมื่อเครื่องเปิดขึ้นมาใหม่ ให้เปิดแอป Ubuntu ที่ปรากฏขึ้นมา ตั้ง Username และ Password สำหรับ Linux ตัวอย่างดังนี้

user : *****

pass: *****

เริ่มการใช้งานสามารถสลับโฟลเดอร์การทำงานไปยังไดรฟ์ Windows เช่น ไดรฟ์ C: หรือ D: ได้ผ่านทางคำสั่ง /mnt/c/ หรือ /mnt/d/ ตัวอย่างเช่น :

cd /mnt/c/Users/YourUsername/Desktop

***จากนี้ไปการทำงานจริงให้พิมพ์คำสั่งทั้งหมดในหน้าต่าง Ubuntu ที่ได้จากการติดตั้ง WSL ครับ

ขั้นตอนที่ 2 : ติดตั้ง Conda และสร้าง Environment โดยการดาวน์โหลดและติดตั้ง Miniconda บน Ubuntu (WSL) ด้วยคำสั่ง

wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

bash Miniconda3-latest-Linux-x86_64.sh

***ระหว่างการติดตั้งให้กด Enter และตอบ yes ตามขั้นตอน แล้วเปิด-ปิดหน้าต่าง Ubuntu ใหม่ 1 รอบ
ขั้นตอนจะบังคับเชื่อมต่อ Conda (รันทีละบรรทัด)
สั่งให้ระบบ Initialize ค่า Conda เข้ากับ Bash:

~/miniconda3/bin/conda init bash

โหลดค่าตั้งใหม่เข้าระบบ:

source ~/.bashrc

ทดสอบความพร้อมของ version :

conda –version

ขั้นตอนยอมรับเงื่อนไขการใช้งาน (Conda ToS)
คัดลอกคำสั่งเหล่านี้ไปวางแล้วกด Enter เพื่อยอมรับเงื่อนไขของช่องหลัก:

conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main

conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r

***ระบบอาจจะถามยืนยัน ให้พิมพ์ y หรือกด Enter ตามที่ระบบแจ้ง

สร้าง Environment ชื่อ flu_analysis และติดตั้งเครื่องมือที่ต้องใช้ทั้งหมด :

conda create -n flu_analysis -c bioconda -c conda-forge minimap2 samtools medaka nanofilt ncbi-datasets-cli -y
conda activate flu_analysis

วิธีตรวจสอบความสำเร็จพิมพ์เพื่อเช็กว่าเครื่องมือพร้อมใช้งาน

minimap2 --version

หลังจากปิดแล้วเปิดใหม่ให้เข้าใช้งาน WSL ผ่านการค้นหาในช่อง windows search คำว่า Ubuntu เข้าใช้งาน

ขั้นตอนที่ 3 : เริ่มต้นทำ Basecalling, Demultiplexing และ Quality Filtering (เริ่มต้นจากไฟล์ FASTQ ดิบล่าสุด) โดยเมื่อได้ไฟล์ FASTQ ดิบจากการรัน Nanopore มาแล้ว หลังจากผ่านการจากการทำ basecalling ด้วย minknow (ตัวอย่างไฟล์ เช่น fastq_runid_2fd7ea28-0720-413a-8a4a-32e5e60300db_0_0.fastq.gz หลายๆ ไฟล์)
1.ขั้นแรก คือการรวมไฟล์ ทำความสะอาดข้อมูล การตรวจสอบคุณภาพ และการกรอง read คุณภาพต่ำออก
เริ่มต้นกลับมาที่โฟลเดอร์งานเดิม
เริ่มจากเข้าผ่าน ubuntu ที่ลงไว้ โดยการคลิกโปรแกรม Ubuntu ที่ลงผ่าน WSL2
พิมพ์

cd /mnt/c/โฟลเดอร์/โฟลเดอร์

เช่น cd /mnt/c/Chim_nanopore/barcode21

จากนั้นพิมพ์ env เพื่อเข้าที่ได้ตั้งไว้

conda activate flu_analysis

เริ่มต้นกระบวนการวิเคราะห์ขั้นตอนที่ 3 เริ่มต้นจาก
#1 รวมไฟล์ FASTQ ดิบทั้งหมดเข้าด้วยกัน (กรณีแยกหลายพาร์ทจากรันเดียวกัน) หรือจากไฟล์รันครั้งเดียวจากเครื่อง nanopore ซึ่งจะอยู่ในโฟลเดอร์ที่มี path ดังนี้ pass/ barcode01 เป็นตัวอย่าง โดยใช้คำสั่งรวมไฟล์ใน folder ในแต่ละ barcode ที่มีข้อมูลนามสกุล *.fastq.gz จำนวนหลายไฟล์ใช้คำสั่งดังนี้

zcat $(find . -name "*.fastq.gz") > all_raw_reads.fastq

*** all_raw_reads.fastq คือชื่อไฟล์ที่เป็น output

#2 ตรวจสอบภาพรวมคุณภาพด้วยการทำ NanoPlot ดูจำนวน read ก่อนทำ filter
เริ่มจากติดตั้ง NanoPlot ใน Environment ปัจจุบัน

pip install NanoPlot
หรือ
conda install -c bioconda nanoplot

*** เลือก conda install -c bioconda nanoplot ได้ดีกว่าเพราะติดตั้งใน Ubuntu
ทำการตรวจสอบว่าระบบมองเห็น NanoPlot หรือไม่

NanoPlot --version
NanoPlot --fastq all_raw_reads.fastq -o nanoplot_raw_out --plots hex

#3 ทำการกรอง read ที่สั้นเกินไป (< 200 bp) และคัดคุณภาพเฉลี่ย (Q-score >= 9) ด้วย SeqKit
ติดตั้ง SeqKit ก่อน:

conda install -c bioconda seqkit

seqkit seq -m 200 -Q 9 all_raw_reads.fastq > filtered.fastq

*****ไฟล์ output คือ filtered.fastq**

สามารถทำการตรวจสอบผล และการแสดงผลด้วยกราฟ (QC & Visualization) จากการกรองข้อมูลด้วย SeqKit:

ข้อมูลที่ผ่านเกณฑ์ควรมีค่า : Median Q-score >10 และมีจำนวน Reads เพียงพอสำหรับการประกอบจีโนม

สคริปต์ในการสร้างกราฟสรุปจำนวน Reads และการกระจายความยาว ด้วยการใช้ Python ทำได้ดังนี้ :

***ให้ทำการติดตั้ง lib ก่อน ด้วยคำสั่งดังนี้ :

pip install matplotlib pandas biopython

pip install biopython matplotlib

***โค้ด python นี้ทำการติดตั้งและรันใน windows ถ้าจะรันใน Ubuntu ก็ให้ติดตั้งในตอนเข้า env ได้

โค้ดตามนี้:

```
from Bio import SeqIO
import matplotlib.pyplot as plt
# โหลดขนาดความยาว Read จากไฟล์ filtered.fastq
read_lengths = [len(rec.seq) for rec in SeqIO.parse("filtered.fastq", "fastq")]
# สร้างกราฟฮิสโตแกรม
plt.figure(figsize=(10, 5))
n, bins, patches = plt.hist(
    read_lengths, bins=40, color="#2ca02c", edgecolor="black", alpha=0.7
)
# วาดเส้นแนวตั้งแสดงจุดตัดความยาวขั้นต่ำ
```

```
plt.axvline(
    x=200, color="r", linestyle="--", label="Minimum Length Cutoff (200 bp)"
)
# เพิ่มตัวเลขจำนวนบนแท่งกราฟแบบแนวตั้ง เพื่อไม่ให้บังกัน
for i in range(len(n)):
  if n[i] > 0:  # แสดงเฉพาะแท่งที่มีข้อมูล
    plt.text(
        bins[i] + (bins[1] - bins[0]) / 2,
        n[i],
        str(int(n[i])),
        ha="center",
        va="bottom",
        rotation=90,  # หมุนตัวเลขเป็นแนวตั้ง
        fontsize=8,
    )
plt.title("Read Length Distribution After QC Filtering", fontsize=12)
plt.xlabel("Read Length (bp)", fontsize=10)
plt.ylabel("Read Count", fontsize=10)
plt.legend()
plt.tight_layout()
# บันทึกภาพกราฟ
plt.savefig("read_length_qc.png", dpi=300)
print("Saved: read_length_qc.png successfully!")
```

*****ผลการทดสอบจะได้ผลการทดสอบชื่อ read_length_qc.png จากการวิเคราะห์ใช้ข้อมูลจาก filtered.fastq**

ขั้นตอนที่ 4 : ขั้นตอนการทำ Minimap2 Screening เพื่อคัดเลือก Best Reference กระบวนการนี้จะช่วยแก้ปัญหาเชื้อกลายพันธุ์สูงหรือยังไม่ทราบสายพันธุ์ โดยนำ Reads จำนวนหนึ่งมาทาบเทียบกับชุด Reference ทางเลือกหลายๆ ตัวพร้อมกัน เพื่อดูว่า Reference ตัวใดให้ผลการจับ (Mapping rate) และความครอบคลุม (Coverage) ดีที่สุด ซึ่งจะดีกว่าการใช้ฐานข้อมูลตายตัว เช่นใน insaflu database ซึ่งใน EPI2ME ซ้ำให้ผลการวิเคราะห์ใน EPI2ME ให้ผลการทดสอบพบ gap และเบส N :

ขั้นตอนที่ 4.1 เตรียมชุดข้อมูล Reference ทางเลือก (Candidate Panel) โดยรวบรวมไฟล์ FASTA ของสายพันธุ์อ้างอิงหลายๆ Clade หรือสายพันธุ์ที่เคยระบาดในอดีตมารวมไว้ในไฟล์เดียวกัน:
ถ้าไฟล์ reference แยกโหลดมาให้รวมก่อนโดยใช้คำสั่ง

cat ชื่อไฟล์1.fasta ชื่อไฟล์2.fasta ชื่อไฟล์3.fasta > candidate_refs.fasta (ชื่อไฟล์ output)

***ถ้ามีฐานข้อมูลไฟล์เดียวกันแล้วก็เปลี่ยนชื่อเป็น **candidate_refs.fasta**

กรณีถ้าต้องการโหลดข้อมูลจาก genbank เพื่อสร้างฐานข้อมูลเชื้อไวรัสที่มียีนครบทั้ง 8 เส้นแบบ complete genome โดยสร้างฐานข้อมูลชื่อ candidate_refs.fasta สามารถทำการเตรียมข้อมูลโดยใช้ script โค้ดด้วย python ดังนี้ :

เปิดคำสั่ง powershell ไปที่ path หรือไปที่ drive เช่นที่ drive E: พิมพ์ E: แล้ว enter หรือไปที่ C: พิมพ์ C: แล้ว enter โดยไม่ใช้ cd

cd " Chim_nanopore\ barcode21"

***ใน powershell หรือ CMD อาจต้องใช้ "" คลุม path ที่จะไปใน version เก่าด้วยก่อนไป path อย่าลืมไปที่ drive ก่อน

#ข้อย่อย 4.1.1: ติดตั้งเครื่องมือ NCBI Datasets CLI

หากยังไม่มีเครื่องมือ datasets บนคอมพิวเตอร์ (รองรับ Windows PowerShell, Linux, และ macOS) สามารถติดตั้งผ่านคำสั่งต่อไปนี้:

สำหรับ Windows (ผ่าน PowerShell):

เปิด PowerShell ในฐานะ Administrator แล้วรันคำสั่งดาวน์โหลดตัวโปรแกรม:

สร้างโฟลเดอร์สำหรับเก็บเครื่องมือและดาวน์โหลด

Invoke-WebRequest -Uri "https://ftp.ncbi.nlm.nih.gov/pub/datasets/command-line/v2/win64/datasets.exe" -OutFile "datasets.exe"

(แนะนำให้เก็บไฟล์ datasets.exe ที่ได้จากคำสั่งไว้ในโฟลเดอร์ที่เซ็ต Path ไว้ หรือเรียกใช้งานจากโฟลเดอร์นั้นโดยตรง)

#ข้อย่อย 4.1.2: ใช้คำสั่งดาวน์โหลดข้อมูล H5N1 ครบ 8 เซกเมนต์

เนื่องจากไวรัสไข้หวัดใหญ่มีจีโนมแยกเป็น 8 เซกเมนต์ (PB2, PB1, PA, HA, NP, NA, M, NS) การดึงข้อมูลให้ครบทั้ง 8 เส้นสมบูรณ์ แนะนำให้ใช้เงื่อนไขกรองเฉพาะข้อมูลสายพันธุ์ที่ระบุว่ามีครบทั้ง 8 เซกเมนต์ (--complete-only และระบุกลุ่มยีน)

โดยใช้ script ภาษา python โดยการกำหนดดังนี้

กำหนดให้เฉพาะยีน H5 ที่ complete ทั้งหมด และมีครบ 8 ยีนที่ complete ด้วย:

กำหนดให้มีครบ 8 ยีนที่ completed ด้วย จากที่เป็น H5N* ทุกข้อมูล :

มีโค้ด python ดังนี้ :

```
from Bio import Entrez
import os

# 1. ตั้งค่าอีเมลของคุณ (ข้อบังคับของ NCBI)
Entrez.email = "your_email@example.com"
# (แนะนำ) ใส่ NCBI API Key หากมี
```

```
# Entrez.api_key = "YOUR_NCBI_API_KEY"

# 2. ปรับ Query ให้ครอบคลุม H5N* ทุกสายพันธุ์ย่อย และเป็น Complete Genome (ครบ 8 ยีน)
# ใช้ H5N* เพื่อดึงทั้ง H5N1, H5N2, H5N6, H5N8 และอื่นๆ และกรองด้วย complete genome

search_term = '(Alphainfluenzavirus influenzae[Organism] AND (H5N* OR H5N1 OR H5N6 OR H5N8) AND (complete genome[Title] OR complete sequence[Title]))'

print("กำลังตรวจสอบจำนวนข้อมูล 'H5N* Complete Genomes' ทั้งหมดจากฐานข้อมูล NCBI...")

# ขั้นตอนที่ 1: ตรวจสอบจำนวนทั้งหมด
handle = Entrez.esearch(db="nucleotide", term=search_term, retmax=0)
search_results = Entrez.read(handle)
handle.close()

total_count = int(search_results["Count"])
print(f"พบข้อมูล H5N* Complete Genomes ทั้งหมด: {total_count:,} รายการ")

if total_count == 0:
    print("ไม่พบข้อมูลที่ตรงกับเงื่อนไข กรุณาตรวจสอบคำค้นหาอีกครั้ง")
else:
    # ขั้นตอนที่ 2: รวบรวมรายชื่อ ID ทั้งหมดแบบ Pagination
    id_list = []
    batch_size_search = 500

    print("กำลังรวบรวมรายชื่อ ID ทั้งหมด...")
    for start in range(0, total_count, batch_size_search):
        handle = Entrez.esearch(
```

```
        db="nucleotide",
        term=search_term,
        retstart=start,
        retmax=batch_size_search
    )
    batch_results = Entrez.read(handle)
    handle.close()
    id_list.extend(batch_results["IdList"])
    print(f"รวบรวม ID แล้ว: {len(id_list)} / {total_count} รายการ")

# ขั้นตอนที่ 3: ดาวน์โหลดไฟล์ FASTA ลงไฟล์เดียว
output_filename = "candidate_refs.fasta"
print(f"\nกำลังเริ่มดาวน์โหลดไฟล์ FASTA ลงในไฟล์: {output_filename}")

fetch_batch_size = 50
with open(output_filename, "w") as output_file:
    for i in range(0, len(id_list), fetch_batch_size):
        batch_ids = id_list[i:i+fetch_batch_size]

        try:
            fetch_handle = Entrez.efetch(
                db="nucleotide",
                id=batch_ids,
                rettype="fasta",
                retmode="text"
            )
            output_file.write(fetch_handle.read())
            fetch_handle.close()
```

```
            except Exception as e:
                print(f"เกิดข้อผิดพลาดในช่วง index {i}: {e}")

            print(f"ดาวน์โหลดสำเร็จแล้ว: {min(i+fetch_batch_size, len(id_list))} / {len(id_list)} รายการ")

    print(f"\nดาวน์โหลดข้อมูล H5N* Complete Genomes ครบทั้งหมดเรียบร้อยแล้ว! บันทึกไฟล์ไว้ที่: {os.path.abspath(output_filename)}")
```

*****เซฟโค้ด python ตั้งชื่อไฟล์ candidate_refs.py ทำการรันจะได้ไฟล์ผลการทดสอบชื่อ candidate_refs.fasta**

ขั้นตอนที่ 4.2 ใช้สคริปต์ Python ตรวจสอบว่า Reads ในไฟล์มีครบทั้ง 8 เซกเมนต์หรือไม่
บางครั้ง Subset reads ที่สกัดมาจาก SeqKit (screening_subset.fastq) อาจมีจำนวนน้อยเกินไปจนทำให้สุ่มเจอเฉพาะเซกเมนต์ที่มีปริมาณไวรัสสูง (High viral titer) แต่เซกเมนต์ปริมาณน้อยอื่นๆ หลุดไป
โดยสามารถใช้สคริปต์ Python ตรวจสอบความครอบคลุมของ Reads ทั้งหมดในไฟล์ filtered.fastq โดยเทียบกับ Reference แบบกว้างๆ ก่อน :

```
from Bio import SeqIO
import subprocess
# นำไฟล์ Reference ที่มีครบ 8 เซกเมนต์แบบมาตรฐาน (เช่น สายพันธุ์อ้างอิงกลาง) มา Map ดูภาพรวมก่อน
# เพื่อเช็กว่า Reads ทั้งหมดวิ่งไปที่เซกเมนต์ไหนบ้าง
print("Checking read distribution across all 8 segments...")
# รันคำสั่ง minimap2 ด่วนเพื่อดูภาพรวมทั้งหมด
cmd = "minimap2 -ax map-ont candidate_refs.fasta filtered.fastq > full_check.sam"
subprocess.run(cmd, shell=True, check=True)
# อ่านผลจาก SAM เพื่อสรุปจำนวน Reads ที่แมพในแต่ละเซกเมนต์
segment_counts = {}
```

```
with open("full_check.sam", "r") as f:
  for line in f:
    if line.startswith("@"):
      continue
    parts = line.strip().split("\t")
    if len(parts) > 2:
      rname = parts[2]  # ชื่อเซกเมนต์ใน Reference
      if rname != "*":
        segment_counts[rname] = segment_counts.get(rname, 0) + 1
print("\n--- Read Distribution per Segment ---")
for seg, count in segment_counts.items():
  print(f"Segment {seg}: {count} reads")
```

***ตั้งชื่อไฟล์ ref_screening.py และรันผ่าน lib ใน Ubuntu ผลการทดสอบจแสดงบน Command Prompt (CMD)

ขั้นตอนที่ 4.3 เมื่อได้ไฟล์ full_check.sam มาแล้ว ขั้นตอนต่อไปคือการแปลงไฟล์ SAM (Sequence Alignment/Map) ให้เป็นรูปแบบไบนารี BAM จัดเรียงตำแหน่ง (Sort) และสร้าง Index เพื่อให้สามารถนำไปวิเคราะห์ความครอบคลุม (Coverage) และสกัดสรุปผลรายเซกเมนต์ สามารถรันชุดคำสั่งผ่านเทอร์มินัล WSL ต่อเนื่องได้ดังนี้ :

#ข้อย่อย 4.3.1 แปลง SAM เป็น BAM, Sort และ Index คำสั่งเหล่านี้จะช่วยย่อขนาดไฟล์ให้เล็กลง จัดเรียงตามตำแหน่งบนจีโนม และเตรียมพร้อมสำหรับดึงสถิติ :

#แปลง SAM เป็น BAM และจัดเรียงตำแหน่ง (Sorted BAM)

samtools view -bS full_check.sam | samtools sort -o full_check.sorted.bam

2. สร้าง Index ให้ไฟล์ BAM

samtools index full_check.sorted.bam

#ข้อย่อย 4.3.2 ตรวจสอบจำนวน Reads ที่แมพได้ในแต่ละเซกเมนต์ ใช้คำสั่ง idxstats เพื่อสรุปดูว่าในแต่ละเซกเมนต์ (PB2, PB1, PA, HA, NP, NA, M, NS) มีจำนวน Read วิ่งไปทาบเทียบเท่าไหร่:

samtools idxstats full_check.sorted.bam > segment_distribution_stats.txt

cat segment_distribution_stats.txt

#ข้อย่อย 4.3.3 สรุปผลลัพธ์และพล็อตภาพด้วย Python (ทางเลือกเสริม)
เพื่อให้เห็นภาพชัดเจนว่าเซกเมนต์ไหนมีปริมาณ Reads หนาแน่นเพียงพอสำหรับการทำ Consensus ต่อไป สามารถใช้สคริปต์ Python สั้นๆ เพื่อสร้างกราฟแท่งสรุปผล:
สร้างไฟล์ชื่อ plot_segments.py หรือรันผ่าน Python ได้ทันที:
เริ่มติดตั้ง lib ใน ubuntu จาก แต่ถ้าติดตั้งใน CMD ปรกติก็ไปใช้ที่ CMD ของ windows ได้ถ้าได้ทำการติดตั้ง lib ไว้แล้ว

คำสั่งติดตั้ง lib ใช้ดังนี้ :

pip install matplotlib pandas biopython

โค้ดสำหรับไฟล์ python ใช้ชื่อไฟล์ plot_segments.py มีดังนี้ :

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd

# 1. อ่านข้อมูลจากไฟล์ idxstats
```

```
data = []
with open('segment_distribution_stats.txt', 'r') as f:
  for line in f:
    parts = line.strip().split('\t')
    if len(parts) == 4 and parts[0] != '*':
      ref_name, length, mapped, unmapped = (
          parts[0],
          int(parts[1]),
          int(parts[2]),
          int(parts[3]),
      )
      data.append(
          {
              'Reference_Name': ref_name,
              'Length': length,
              'Mapped_Reads': mapped,
              'Unmapped_Reads': unmapped,
          }
      )

df = pd.DataFrame(data)

if df.empty:
  print(
      '[WARNING] ไม่พบข้อมูลในไฟล์ segment_distribution_stats.txt หรือไม่มี'
      ' Segment ใดถูกแมพเลย!'
  )
else:
```

```
# เรียงลำดับจากจำนวน Mapped Reads มากไปน้อยสำหรับทำรายงาน Text
df_sorted = df.sort_values('Mapped_Reads', ascending=False)

# 2. ส่งออกข้อมูลชื่อเต็มทั้งหมดเป็นไฟล์ Text สรุปผล
report_filename = 'segment_summary_report.txt'
with open(report_filename, 'w', encoding='utf-8') as out:
  out.write('=== Influenza A Segment Coverage & Mapped Reads Summary ===\n')
  out.write(
      f"{'Reference Name':<55} | {'Length (bp)':<12} | {'Mapped Reads':<12}\n"
  )
  out.write('-' * 85 + '\n')
  for index, row in df_sorted.iterrows():
    out.write(
        f"{row['Reference_Name']:<55} | {row['Length']:<12,}"
        f" | {row['Mapped_Reads']:<12,}\n"
    )
print(f'>>> บันทึกรายงานสรุปสำเร็จ: {report_filename} <<<')

# 3. กรองเฉพาะตัวที่มี Mapped Reads > 0 สำหรับนำไปทำกราฟ
df_filtered = df[df['Mapped_Reads'] > 0].copy()

if df_filtered.empty:
  print(
      '[WARNING] มีข้อมูลใน Text Report แต่ Mapped Reads เป็น 0 ทุกเซกเมนต์!'
  )
else:
  # เรียงลำดับจากน้อยไปมากสำหรับกราฟแท่งแนวนอน
  df_filtered = df_filtered.sort_values('Mapped_Reads', ascending=True)
```

```
# หากมีผลลัพธ์เยอะเกินไป ให้แสดงสูงสุด 30 อันดับแรก
if len(df_filtered) > 30:
  df_filtered = df_filtered.tail(30)

# ตัดชื่อให้สั้นลงเฉพาะบนกราฟ เพื่อความสะอาด (ชื่อเต็มถูกเก็บไว้ใน Text Report เรียบร้อยแล้ว)
plot_names = []
for name in df_filtered['Reference_Name']:
  short_name = name.split('|')[-1]
  if len(short_name) > 35:
    short_name = short_name[:32] + '...'
  plot_names.append(short_name)
df_filtered['Plot_Name'] = plot_names

# พล็อตราฟ
plt.figure(figsize=(11, max(4, len(df_filtered) * 0.35)))
plt.barh(
    df_filtered['Plot_Name'],
    df_filtered['Mapped_Reads'],
    color='#1f77b4',
    edgecolor='black',
    alpha=0.8,
)

plt.title(
    'Mapped Reads Distribution per Reference Segment', fontsize=12, pad=15
)
plt.xlabel('Number of Mapped Reads', fontsize=10)
```

```
plt.ylabel('Reference Sequence / Segment', fontsize=10)
plt.tight_layout()

plt.savefig('segment_coverage_distribution.png', dpi=150)
print(
    '>>> บันทึกภาพกราฟสำเร็จ: segment_coverage_distribution.png <<<'
)
```

*****ผลการทดสอบจะแสดงกราฟแท่งในไฟล์ชื่อ segment_coverage_distribution.png**

ขั้นตอนที่ 5 : เพื่อเชื่อมโยงชื่อสั้น (Ref_ID) ที่ได้จากผลการทำ Mapping (จากขั้นตอน SAM (Sequence Alignment/Map)) กลับเข้ากับชื่อเต็มและรายละเอียด (Full Header) ในไฟล์ฐานข้อมูลตั้งต้นอย่าง candidate_refs.fasta (หรือไฟล์ Reference ที่ใช้งานอยู่) พร้อมทั้งจัดเรียงลำดับจากจำนวน Mapped Reads จาก มากไปหาน้อย โดยใช้สคริปต์ Python ด้านล่างนี้จัดการ
สคริปต์นี้จะใช้ Bio.SeqIO อ่านหัวข้อ (Header) เต็มจากไฟล์ FASTA มาจับคู่กับผลลัพธ์จาก idxstats แล้วบันทึกออกเป็นทั้งรายงานข้อความ (matched_segment_summary.txt) และกราฟแท่งแนวนอน
โค้ดสำหรับจับคู่ชื่อและจัดเรียงรายงานโดยตั้งชื่อไฟล์ดังนี้ match_references.py ซึ่งมี script โค้ด python ดังนี้

```
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd
from Bio import SeqIO
# 1. กำหนดชื่อไฟล์ฐานข้อมูลเริ่มต้น (สามารถเปลี่ยนชื่อไฟล์ FASTA ได้ตามจริง เช่น candidate_refs.fasta หรือ reference_all_8segments.fasta)
fasta_filename = 'candidate_refs.fasta'
# อ่านชื่อเต็ม (Full Header) จากไฟล์ FASTA เก็บใส่ Dictionary
ref_full_names = {}
```

```
try:
  for record in SeqIO.parse(fasta_filename, 'fasta'):
    # record.id คือชื่อไอดีสั้น, record.description คือข้อความส่วนหัวทั้งหมดใน FASTA
    ref_full_names[record.id] = record.description
  print(f'โหลดฐานข้อมูลจาก {fasta_filename} สำเร็จ!')
except FileNotFoundError:
  print(
      f'[WARNING] ไม่พบไฟล์ {fasta_filename} จะใช้ชื่อ ID ดั้งเดิมแทน'
      ' (ตรวจสอบชื่อไฟล์ใหม่อีกครั้ง)'
  )
# 2. อ่านข้อมูลสถิติจาก segment_distribution_stats.txt
data = []
with open('segment_distribution_stats.txt', 'r') as f:
  for line in f:
    parts = line.strip().split('\t')
    if len(parts) == 4 and parts[0] != '*':
      ref_id, length, mapped, unmapped = (
          parts[0],
          int(parts[1]),
          int(parts[2]),
          int(parts[3]),
      )
      # ดึงชื่อเต็มจาก FASTA หากหาไม่เจอให้ใช้ ref_id เดิม
      full_name = ref_full_names.get(ref_id, ref_id)
      data.append(
          {
              'Ref_ID': ref_id,
              'Full_Name': full_name,
```

```
            'Length': length,
            'Mapped_Reads': mapped,
            'Unmapped_Reads': unmapped,
        }
    )
df = pd.DataFrame(data)
if df.empty:
  print('[WARNING] ไม่พบข้อมูลในไฟล์ segment_distribution_stats.txt!')
else:
  # เรียงลำดับจาก Mapped Reads มากไปหาน้อย (Highest to Lowest)
  df_sorted = df.sort_values('Mapped_Reads', ascending=False)
  # 3. บันทึกรายงานสรุปเป็นไฟล์ Text (เรียงจากมากไปน้อย)
  report_filename = 'matched_segment_summary.txt'
  with open(report_filename, 'w', encoding='utf-8') as out:
    out.write(
      '=== Influenza A Mapped Reads Matched with Candidate FASTA Headers'
      ' ===\n'
    )
    out.write(
      f"{'Reference ID / Full Header Description':<75} | {'Length (bp)':<12}"
      f" | {'Mapped Reads':<12}\n"
    )
    out.write('-' * 105 + '\n')
    for _, row in df_sorted.iterrows():
      out.write(
        f"{row['Full_Name']:<75} | {row['Length']:<12,}"
        f" | {row['Mapped_Reads']:<12,}\n"
      )
```

```
print(f'>>> บันทึกรายงานสรุปสำเร็จ: {report_filename} <<<')
# แสดงผล 10 อันดับแรกที่จับคู่ได้ดีที่สุดออกทางหน้าจอ
print('\n--- Top 10 Mapped Reference Candidates ---')
for _, row in df_sorted.head(10).iterrows():
  print(f'[{row["Mapped_Reads"]:,} reads] -> {row["Full_Name"]}')
# 4. กรองเฉพาะตัวที่มี Reads > 0 สำหรับนำไปพล็อตเป็นกราฟ
df_filtered = df[df['Mapped_Reads'] > 0].copy()
if not df_filtered.empty:
  # สำหรับกราฟแนวนอน ให้เรียงจากน้อยไปมากเพื่อให้ค่ามากที่สุดอยู่ด้านบนสุดของกราฟ
  df_filtered = df_filtered.sort_values('Mapped_Reads', ascending=True)
  # หากมีผลลัพธ์เยอะเกินไป ให้แสดงเฉพาะ 25 อันดับสูงสุด
  if len(df_filtered) > 25:
    df_filtered = df_filtered.tail(25)
  # ตัดชื่อให้สั้นลงพอดีกับกราฟ
  plot_labels = []
  for _, row in df_filtered.iterrows():
    name = row['Full_Name']
    if len(name) > 45:
      name = name[:42] + '...'
    plot_labels.append(name)
  df_filtered['Plot_Label'] = plot_labels
  # สร้างกราฟ
  plt.figure(figsize=(12, max(5, len(df_filtered) * 0.35)))
  plt.barh(
      df_filtered['Plot_Label'],
      df_filtered['Mapped_Reads'],
      color='#2ca02c',
      edgecolor='black',
```

```
    alpha=0.8,
)
plt.title(
    'Mapped Reads per Reference (Matched with Candidate Database)',
    fontsize=12,
    pad=15,
)
plt.xlabel('Number of Mapped Reads', fontsize=10)
plt.ylabel('Reference Headers', fontsize=10)
plt.tight_layout()
plt.savefig('matched_segments_plot.png', dpi=150)
print(
    '>>> บันทึกภาพกราฟสำเร็จ: matched_segments_plot.png (จัดเรียงเรียบร้อย)'
    ' <<<'
)
```

*****ผลการทดสอบจะแสดงกราฟแท่งระหว่างลำดับ ref จากฐานข้อมูล candidate_refs.fasta และจำนวน read ที่พบ โดยผลแสดงไฟล์ชื่อ matched_segments_plot.png และผลข้อมูลในไฟล์ชื่อ matched_segment_summary.txt**

ขั้นตอนที่ 6 : การทำ Alignment & Coverage Analysis (การทาบเทียบและวิเคราะห์ความครอบคลุมเชิงลึก)
6.1 การทำ Full Alignment กับ Reference ที่คัดเลือกแล้วให้นำไฟล์ FASTA ของ Reference ที่ดีที่สุดที่ผู้ทดสอบเลือกเองจากลำดับในไฟล์ **matched_segment_summary.txt** นำมาเลือกลำดับนิวคลีโอไทด์จากไฟล์ **candidate_refs.fasta** เพื่อรวบรวมเป็นไฟล์ฐานข้อมูลอ้างอิง โดยการบันทึกเป็นชื่อไฟล์ว่า **selected_reference.fasta** มาใช้เป็น Reference หลักในการรัน Alignment แบบเต็มรูปแบบกับไฟล์ Reads ที่ผ่านการกรองคุณภาพแล้ว (filtered.fastq) ในขั้นตอนนี้ปฏิบัติได้ดังนี้

6.1.1 สร้าง Index ของ Reference ที่เลือก

minimap2 -d selected_reference.mmi selected_reference.fasta

6.1.2 รัน Minimap2 แบบละเอียดสำหรับ Nanopore reads (-ax map-ont) ได้ดังนี้

minimap2 -ax map-ont selected_reference.mmi filtered.fastq > final_alignment.sam

6.1.3 แปลง SAM เป็น Sorted BAM และสร้าง Index

samtools view -bS final_alignment.sam | samtools sort -o final_alignment.sorted.bam
samtools index final_alignment.sorted.bam

6.1.4 ตรวจสอบสถิติภาพรวมของการ Mapping (Mapping rate, % mapped)

samtools flagstat final_alignment.sorted.bam

6.2 คำนวณค่าความลึก (Coverage Depth) รายตำแหน่ง เพื่อให้ทราบว่าแต่ละตำแหน่งบนเซกเมนต์มี Reads มาทาบกี่ Reads (Coverage Depth) และมีบริเวณไหนที่ Read ขาดหายไป (Dropout) หรือไม่ โดยใช้คำสั่ง samtools depth ดังนี้
สกัดข้อมูลความลึกรายตำแหน่งเก็บไว้เป็นไฟล์ข้อความ

samtools depth -a final_alignment.sorted.bam > coverage_depth.txt

6.3 สคริปต์ Python สำหรับวิเคราะห์และแสดงกราฟ Coverage Profile (รายเซกเมนต์)
เนื่องจากไวรัสไข้หวัดใหญ่มี 8 เซกเมนต์ การดูกราฟความครอบคลุม (Coverage Depth Profile) ของแต่ละเซกเมนต์จะช่วยให้เราเห็นภาพชัดเจนว่า เซกเมนต์ใดมีความสมบูรณ์ครบ 100% (Full-length coverage) สามารถสร้างไฟล์ชื่อ generate_coverage_html.py ด้วยโค้ดด้านล่างนี้

```
import base64
import io
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import pandas as pd
from Bio import SeqIO

# 1. โหลดชื่อเต็ม (Full Header Description) จากไฟล์ selected_reference.fasta
fasta_filename = 'selected_reference.fasta'
ref_full_names = {}
try:
  for record in SeqIO.parse(fasta_filename, 'fasta'):
    # เก็บข้อมูลโดยใช้ record.id เป็น Key และ record.description เป็นชื่อเต็ม
    ref_full_names[record.id] = record.description
  print(f'โหลดฐานข้อมูล Reference จาก {fasta_filename} สำเร็จ!')
except FileNotFoundError:
  print(
      f'[WARNING] ไม่พบไฟล์ {fasta_filename} จะใช้ชื่อ ID ดั้งเดิมแทน'
      ' (ตรวจสอบชื่อไฟล์ใหม่อีกครั้ง)'
  )
# 2. อ่านข้อมูลจากไฟล์ coverage_depth.txt
print('กำลังอ่านข้อมูล Coverage Depth...')
data = []
with open('coverage_depth.txt', 'r') as f:
  for line in f:
    parts = line.strip().split('\t')
    if len(parts) == 3:
```

```
        ref_id, pos, depth = parts[0], int(parts[1]), int(parts[2])
        data.append({'Reference_ID': ref_id, 'Position': pos, 'Depth': depth})
df = pd.DataFrame(data)
if df.empty:
    print('[WARNING] ไม่พบข้อมูลในไฟล์ coverage_depth.txt!')
else:
    references = df['Reference_ID'].unique()
    print(
        f'พบทั้งหมด {len(references)} เซกเมนต์ กำลังสร้างรายงาน HTML แบบชื่อเต็ม...'
    )
    # โครงสร้าง HTML พร้อม CSS ตกแต่งให้อ่านง่าย
    html_content = """
    <!DOCTYPE html>
    <html lang="th">
    <head>
        <meta charset="UTF-8">
        <title>Influenza A Full Coverage Depth Report</title>
        <style>
            body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; margin: 30px;
background-color: #f4f7f6; color: #333; }
            h1 { color: #2c3e50; text-align: center; margin-bottom: 5px; }
            .subtitle { text-align: center; color: #7f8c8d; margin-bottom: 30px; font-size: 14px; }
            .segment-card { background: #ffffff; border-radius: 10px; box-shadow: 0 4px 6px
rgba(0,0,0,0.05); margin-bottom: 25px; padding: 20px; border-left: 5px solid #3498db; }
            .segment-title { font-size: 16px; font-weight: bold; color: #2980b9; margin-bottom: 12px;
display: flex; justify-content: space-between; align-items: center; flex-wrap: wrap; gap: 10px; }
            .badge { background: #e8f4f8; color: #2980b9; padding: 4px 10px; border-radius: 12px;
font-size: 13px; font-weight: normal; }
```

```
            .chart-container { text-align: center; }
            img { max-width: 100%; height: auto; border-radius: 6px; }
        </style>
    </head>
    <body>
        <h1>Influenza A - Coverage Depth Analysis Report (Full Headers)</h1>
        <div class="subtitle">รายงานสรุปความลึกและความครอบคลุมของจีโนมไวรัสไข้หวัดใหญ่ พร้อมชื่อเต็มจากฐานข้อมูลอ้างอิง</div>
    """
  # 3. วนลูปสร้างกราฟรายเซกเมนต์และดึงชื่อเต็มมาแสดง
  for ref_id in references:
    df_seg = df[df['Reference_ID'] == ref_id]
    # ดึงชื่อเต็มจาก Dictionary (ถ้าไม่มี ใช้ ref_id แทน)
    full_name = ref_full_names.get(ref_id, ref_id)
    # คำนวณสถิติเบื้องต้น
    mean_depth = df_seg['Depth'].mean()
    max_depth = df_seg['Depth'].max()
    # สร้างกราฟด้วย Matplotlib
    plt.figure(figsize=(11, 3.2))
    plt.plot(
        df_seg['Position'],
        df_seg['Depth'],
        color='#1f77b4',
        linewidth=1.2,
        alpha=0.9,
    )
    plt.fill_between(
        df_seg['Position'], df_seg['Depth'], color='#1f77b4', alpha=0.25
```

```
)
plt.axhline(
    y=mean_depth,
    color='#e74c3c',
    linestyle='--',
    linewidth=1.2,
    label=f'Mean Depth: {mean_depth:.1f}x (Max: {max_depth:,}x)',
)
plt.title(
    f'Coverage Profile: {ref_id}', fontsize=11, fontweight='bold'
)  # ใช้ ID สั้นบนกราฟเพื่อความสะอาด
plt.xlabel('Genome Position (bp)', fontsize=9)
plt.ylabel('Sequencing Depth', fontsize=9)
plt.legend(loc='upper right', fontsize=9)
plt.grid(True, linestyle=':', alpha=0.6)
plt.tight_layout()
# แปลงภาพกราฟเป็น Base64
buffer = io.BytesIO()
plt.savefig(buffer, format='png', dpi=150)
buffer.seek(0)
img_base64 = base64.b64encode(buffer.getvalue()).decode('utf-8')
plt.close()
# แทรกข้อมูลการ์ดลงใน HTML โดยแสดงชื่อเต็มแบบครบถ้วน
html_content += f"""
    <div class="segment-card">
        <div class="segment-title">
            <span>Reference: {full_name}</span>
            <span class="badge">Mean Depth: {mean_depth:.1f}x | Max: {max_depth:,}x</span>
```

```
            </div>
            <div class="chart-container">
                <img src="{img_base64}" alt="{ref_id}">
            </div>
        </div>
        """
    html_content += """
      </body>
      </html>
      """
    # 4. บันทึกไฟล์ HTML ผลลัพธ์
    output_html = 'coverage_report.html'
    with open(output_html, 'w', encoding='utf-8') as f:
      f.write(html_content)
    print(f'\n>>> สร้างรายงาน HTML พร้อมชื่อเต็มสำเร็จ: {output_html} <<<')
```

***ผลการวิเคราะห์จะแสดงกราฟ Coverage Profile รายเซกเมนต์ ด้วยไฟล์ชื่อ **coverage_report.html**

ขั้นตอนที่ 7 : การสร้าง Consensus Generation & Variant Calling ด้วย Medaka

7.1 ตรวจสอบและติดตั้ง Medaka (หากยังไม่มีใน Environment)

เปิดเทอร์มินัล WSL และเปิดใช้งาน Conda Environment แล้วตรวจสอบว่ามี Medaka หรือไม่

ถ้ายังไม่ทำงานใน env การวิเคราะห์ flu_analysis ให้ใช้คำสั่งนี้ และตรวจสอบว่าสามารถใช้ Medaka ได้หรือยัง

conda activate flu_analysis

medaka –version

7.2 รันคำสั่งสร้าง Consensus ด้วย Medaka

***การทำงาน Medaka จะใช้โครงข่ายประสาทเทียม (Neural Network) เพื่อแก้ไขข้อผิดพลาด (Error correction) และสร้างลำดับเบสฉันทามติที่แม่นยำสูงจากไฟล์ BAM และ Reference ตั้งต้น

7.2.1 ขั้นตอนขัดเกลาและสร้าง Consensus Sequence ด้วย Medaka โดยใช้คำสั่ง medaka_consensus โดยระบุโมเดลให้ตรงกับรุ่นของ Flow Cell และ Kit ที่ใช้รันจริง (เช่น r1041_e82_400bps_hac หรือ _sup สำหรับโหมด Super Accuracy)

รัน Medaka Consensus เริ่มจากติดตั้งและเชื่อมโยงแพ็กเกจ bcftools และ gsl ให้ถูกต้องผ่าน Conda ด้วยคำสั่งนี้

conda install -c bioconda -c conda-forge bcftools gsl --force-reinstall

medaka_consensus -i filtered.fastq -d selected_reference.fasta -o medaka_out -m r1041_e82_400bps_sup_v5.2.0 -t 8

#คำอธิบายพารามิเตอร์:

-i: ไฟล์ FASTQ ของ Reads ที่ผ่านการกรองแล้ว

-d: ไฟล์ FASTA ของ Reference ที่คัดเลือกไว้ดีที่สุด

-o: โฟลเดอร์ปลายทางสำหรับเก็บผลลัพธ์ (medaka_out)

-m: โมเดลของ Nanopore sequencing (ปรับเปลี่ยนได้ตาม Flow cell เช่น r1041_e82_400bps_sup หากใช้โหมดความแม่นยำสูงพิเศษ)

-t: จำนวน Threads ที่ใช้ประมวลผล (เช่น 8 เธรด)

*****เมื่อรันเสร็จเรียบร้อย ผลลัพธ์หลักจะถูกเก็บไว้ในไฟล์ชื่อ medaka_out/consensus.fasta ซึ่งจะประกอบด้วยลำดับจีโนมครบทั้ง 8 เซกเมนต์**

7.3 ตรวจสอบความสมบูรณ์ของไฟล์ Consensus ด้วย Python เพื่อให้มั่นใจว่าเซกเมนต์ทั้ง 8 ถูกสร้างขึ้นครบถ้วนและมีความยาวถูกต้องตามธรรมชาติของไวรัสไข้หวัดใหญ่ สามารถใช้สคริปต์ตรวจสอบด่วนได้ดังนี้

สร้างและรันสคริปต์ชื่อ check_consensus.py

```
from Bio import SeqIO
consensus_file = "medaka_out/consensus.fasta"
print("=== รายงานสรุปผล Consensus Sequences (8 Segments) ===")
try:
  count = 0
  for record in SeqIO.parse(consensus_file, "fasta"):
    count += 1
    print(f"Segment {count}: {record.id} | ความยาว: {len(record.seq):,} bp")
  print(
      f"\n>>> ตรวจพบทั้งหมด {count} เซกเมนต์ในไฟล์ Consensus เรียบร้อยสมบูรณ์"
      "ครับ <<<"
  )
except FileNotFoundError:
  print(f"[WARNING] ไม่พบไฟล์ {consensus_file} กรุณาตรวจสอบเส้นทางโฟลเดอร์อีกครั้ง")
```

*****ผลการรันจะแสดงผลใน CMD**

7.4 การทดสอบ Consensus QC & Final Reporting ด้วยการตรวจสอบคุณภาพลำดับ Consensus (Consensus Quality & Gap Check) ก่อนนำไฟล์ Consensus (medaka_out/consensus.fasta) ไปใช้งาน ควรตรวจสอบว่าในแต่ละเซกเมนต์มีตำแหน่งเบสที่ขาดหายหรือไม่ (โดยเช็คสัดส่วนเบส N หรือช่องว่าง) เพื่อให้แน่ใจว่าลำดับจีโนมมีความสมบูรณ์ สามารถใช้สคริปต์ Python ตรวจสอบปริมาณเบส N ในแต่ละเซกเมนต์ได้ทันที

***เกณฑ์มาตรฐาน คือ เซกเมนต์ที่ดีควรมีสัดส่วนเบส N ต่ำมาก หรือเป็น 0% ในบริเวณ Coding Region เพื่อให้มั่นใจว่าเป็น Complete Coding Sequence (CDS)

```
from Bio import SeqIO
consensus_file = 'medaka_out/consensus.fasta'
print('=== รายงานตรวจสอบคุณภาพลำดับ Consensus (N Content & Length) ===')
for record in SeqIO.parse(consensus_file, 'fasta'):
  seq_str = str(record.seq)
  length = len(seq_str)
  n_count = seq_str.upper().count('N')
  n_percent = (n_count / length) * 100 if length > 0 else 0
  print(
      f'Segment: {record.id:<15} | ความยาว: {length:,} bp | '
      f'Ambiguous (N): {n_count} ({n_percent:.2f}%)'
  )
```

*****ตั้งชื่อไฟล์ Final_Reporting.py ผลการรันแสดงใน CMD**

ขั้นตอนที่ 8 : การตรวจสอบความสมบูรณ์ของโครงสร้าง (Structural Integrity Check) เพื่อป้องกันปัญหาการขาดท่อน (Truncation/Deletions) หรือการต่อสายผิดปกติ (Chimeras/Mis-assemblies) ถือเป็นขั้นตอน QC ที่สำคัญมากสำหรับไวรัสที่มีโครงสร้างเป็นเซกเมนต์อย่าง Influenza A (H5N1) เนื่องจากเซกเมนต์ของไวรัสไข้หวัดใหญ่แต่ละตัวมีความยาวมาตรฐานที่แน่นอน เราสามารถเขียนสคริปต์ Python เพื่อเปรียบเทียบความยาวของ Consensus (medaka_out/consensus.fasta) กับ Reference ตั้งต้น (selected_reference.fasta) รวมถึงตรวจสอบปริมาณเบส N ได้อย่างแม่นยำ

***สคริปต์ตรวจสอบโครงสร้างและความสมบูรณ์ (Structural & Length QC) สร้างและรันสคริปต์ Python ในโฟลเดอร์ชื่อ Structural_Integrity_Check.py

```
from Bio import SeqIO
from datetime import datetime
ref_file = "selected_reference.fasta"
consensus_file = "medaka_out/consensus.fasta"
```

```
# 1. โหลดข้อมูลความยาว Reference
ref_lengths = {}
for record in SeqIO.parse(ref_file, "fasta"):
    ref_lengths[record.id] = len(record.seq)
# 2. โหลดข้อมูล Consensus
cons_records = {record.id: record for record in SeqIO.parse(consensus_file, "fasta")}
# 3. ประมวลผลข้อมูลแต่ละ Segment
table_rows = ""
for ref_id, ref_len in ref_lengths.items():
    if ref_id in cons_records:
        rec = cons_records[ref_id]
        cons_len = len(rec.seq)
        n_count = str(rec.seq).upper().count('N')
        n_percent = (n_count / ref_len) * 100 if ref_len > 0 else 0
        # กำหนดสถานะและสี badge
        status = "Normal (สมบูรณ์ดี)"
        badge_class = "badge-success"
        if cons_len < ref_len * 0.85:
            status = " ⚠ Truncated (สั้นเกินไป)"
            badge_class = "badge-warning"
        elif cons_len > ref_len * 1.15:
            status = " ⚠ Too Long (ยาวผิดปกติ)"
            badge_class = "badge-warning"
        elif n_count > (ref_len * 0.05):
            status = " ⚠ High Gaps (N สูง)"
            badge_class = "badge-warning"
        table_rows += f"""
        <tr>
```

```
            <td><strong>{ref_id}</strong></td>
            <td>{ref_len:,} bp</td>
            <td>{cons_len:,} bp</td>
            <td>{n_count:,} ({n_percent:.2f}%)</td>
            <td><span class="badge {badge_class}">{status}</span></td>
        </tr>
        """
    else:
        table_rows += f"""
        <tr>
            <td><strong>{ref_id}</strong></td>
            <td>{ref_len:,} bp</td>
            <td colspan="3"><span class="badge badge-danger">❌ Missing
Segment</span></td>
        </tr>
        """
# 4. สร้างโครงสร้าง HTML Dashboard
html_content = f"""<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>H5N1 Structural & Consensus QC Report</title>
    <style>
        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f6f9;
            color: #333;
```

```
    margin: 0;
    padding: 20px;
}}
.container {{
    max-width: 1000px;
    margin: auto;
    background: #fff;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.05);
}}
h1 {{
    color: #2c3e50;
    font-size: 24px;
    border-bottom: 2px solid #eaeaea;
    padding-bottom: 15px;
    margin-top: 0;
}}
.meta-info {{
    background: #f8f9fa;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 25px;
    font-size: 14px;
    color: #555;
}}
table {{
    width: 100%;
```

```
    border-collapse: collapse;
    margin-top: 10px;
}}
th, td {{
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid #e1e1e1;
}}
th {{
    background-color: #34495e;
    color: #ffffff;
    font-weight: 600;
}}
tr:hover {{
    background-color: #f8f9fa;
}}
.badge {{
    padding: 6px 12px;
    border-radius: 20px;
    font-size: 12px;
    font-weight: 600;
    display: inline-block;
}}
.badge-success {{
    background-color: #d4edda;
    color: #155724;
}}
.badge-warning {{
```

```
            background-color: #fff3cd;
            color: #856404;
        }}
        .badge-danger {{
            background-color: #f8d7da;
            color: #721c24;
        }}
        .footer {{
            margin-top: 30px;
            text-align: center;
            font-size: 12px;
            color: #888;
            border-top: 1px solid #eaeaea;
            padding-top: 15px;
        }}
    </style>
</head>
<body>
    <div class="container">
        <h1> 🧬 H5N1 Nanopore Pipeline: Structural & Consensus QC Report</h1>
        <div class="meta-info">
            <strong>Analysis Date:</strong> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}<br>
            <strong>Reference File:</strong> <code>{ref_file}</code><br>
            <strong>Consensus File:</strong> <code>{consensus_file}</code><br>
            <strong>Pipeline Model:</strong> Medaka (r1041_e82_400bps_sup_v5.2.0)
        </div>
        <table>
            <thead>
```

```
                <tr>
                    <th>Segment ID</th>
                    <th>Ref Length</th>
                    <th>Consensus Length</th>
                    <th>Ambiguous (N) Count</th>
                    <th>Structural Status</th>
                </tr>
            </thead>
            <tbody>
                {table_rows}
            </tbody>
        </table>
        <div class="footer">
            Generated automatically by Flu Analysis Pipeline (WSL2 / Bioinformatics Workflow)
        </div>
    </div>
</body>
</html>
"""
# บันทึกไฟล์ HTML
with open("structural_qc_report.html", "w", encoding="utf-8") as f:
    f.write(html_content)
print(">>> สร้างไฟล์รายงาน HTML สำเร็จเรียบร้อยแล้ว: structural_qc_report.html <<<")
```

*****ไฟล์แสดงผลการทดสอบแสดงในไฟล์ชื่อ structural_qc_report.html**

ขั้นตอนที่ 9 : การทดสอบ Typing และ Clade Assignment (ระบุสายพันธุ์และ Clade) ระบุจำแนก Subtype และ Clade ของไวรัส เช่น Clade 2.3.4.4b

9.1 ขั้นตอนการค้นหายีนเซกเมนต์ HA (Hemagglutination) จากไฟล์ Consensus ทั้ง 8 เซกเมนต์หลังการประกอบยีน เพื่อดึงลำดับยีน HA ออกมาเป็นไฟล์เฉพาะยีน HA ชื่อ HA_segment_consensus.fasta และการเตรียมไฟล์ให้อยู่ในรูปแบบที่พร้อมสำหรับการวิเคราะห์ Clade ไฟล์สำหรับการรันทดสอบตั้งชื่อไฟล์คือ HA_segment_consensus.py

```
from Bio import SeqIO
input_fasta = "medaka_out/consensus.fasta"
output_ha = "HA_segment_consensus.fasta"
ha_found = False
for record in SeqIO.parse(input_fasta, "fasta"):
    # ตรวจสอบชื่อเซกเมนต์ HA จาก Header หรือความยาวประมาณ 1,700 - 1,800 bp
    if "HA" in record.id.upper() or (1700 <= len(record) <= 1800):
        SeqIO.write(record, output_ha, "fasta")
        print(f"☑ คัดลอกเซกเมนต์ HA สำเร็จ: {record.id} (ความยาว: {len(record)} bp) บันทึกที่ {output_ha}")
        ha_found = True
        break
if not ha_found:
    print("⚠ ไม่พบคำว่า HA ชัดเจน กรุณาตรวจสอบรายชื่อ Contig ทั้งหมด:")
    for record in SeqIO.parse(input_fasta, "fasta"):
        print(f" - ID: {record.id} | ความยาว: {len(record)} bp")
```

*****ไฟล์ลำดับนิวคลีโอไทด์ยีน HA ถูกค้นหาจากไฟล์ consensus.fasta จากการ assembly ที่สมบูรณ์ และยีน HA ที่ถูกคัดเลือกจะถูกบันทึกในชื่อไฟล์ HA_segment_consensus.fasta**

9.2 ขั้นตอนการวิธีวิเคราะห์ผ่าน Nextclade โดยการรันผ่าน Command Line (CLI) ใน WSL ผ่านเทอร์มินัลใน WSL โดยตรง (ไม่ต้องผ่านหน้าเว็บ) Nextclade ก็มีเครื่องมือแบบ CLI ให้ติดตั้งผ่าน Conda
ติดตั้ง nextclade ใน conda environment (ถ้าต้องการ)

conda install -c bioconda nextclade -y

รันวิเคราะห์ผ่านคำสั่ง cli (ตัวอย่าง) วิธีใช้งาน Nextclade CLI ที่ถูกต้อง
1. ค้นหา Dataset ของ H5 ที่มีในระบบ (จะพบชื่อ Dataset ของ H5 HA เช่น community/moncla-lab/iav-h5/ha/all-clades) เพื่อดูฐานข้อมูลทั้งหมดเพื่อจะนำไปเลือกใช้

nextclade dataset list --search h5

2. รันคำสั่งวิเคราะห์โดยระบุชื่อ Dataset จริง
เริ่มจากดาวน์โหลด Dataset ของ H5 มาเก็บไว้ในเครื่องก่อน

nextclade dataset get --name 'community/moncla-lab/iav-h5/ha/all-clades' --output-dir 'h5_dataset'

รันคำสั่งวิเคราะห์โดยระบุชื่อ Dataset ที่ใช้งานจริง

nextclade run --input-dataset h5_dataset HA_segment_consensus.fasta --output-all nextclade_out/

3. สร้างและเขียนสคริปต์สร้างรายงาน HTML สำหรับผลที่รันได้จาก Nextclade รันโดยใช้คำสั่งนี้เพื่อสร้างไฟล์สคริปต์ generate_nextclade_html.py

import pandas as pd
from datetime import datetime

```
import os
tsv_file = "nextclade_out/nextclade.tsv"
output_html = "nextclade_report.html"
if not os.path.exists(tsv_file):
    print(f"[ERROR] ไม่พบไฟล์ {tsv_file} กรุณาตรวจสอบว่ารัน Nextclade สำเร็จแล้วหรือยัง")
else:
    df = pd.read_csv(tsv_file, sep='\t')
    rows_html = ""
    for idx, row in df.iterrows():
        seq_name = row.get('seqName', 'N/A')
        clade = row.get('clade', 'N/A')
        qc_status = row.get('qc.overallStatus', 'N/A')
        mutations = row.get('totalMutations', 0)
        deletions = row.get('totalDeletions', 0)
        missing = row.get('totalMissing', 0)
        substitutions = row.get('substitutions', 'N/A')
        # กำหนดสี Badge ตามสถานะ QC
        if qc_status == "good":
            qc_badge = "badge-success"
        elif qc_status == "mediocre":
            qc_badge = "badge-warning"
        else:
            qc_badge = "badge-danger"
        rows_html += f"""
        <tr>
            <td><strong>{seq_name}</strong></td>
            <td><span class="badge badge-clade">{clade}</span></td>
            <td><span class="badge {qc_badge}">{qc_status}</span></td>
```

```
            <td>{mutations}</td>
            <td>{deletions}</td>
            <td>{missing}</td>
            <td style="font-family: monospace; font-size: 11px; max-width: 350px; word-break:
break-all;">{substitutions}</td>
        </tr>
        """
    html_content = f"""<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>H5N1 Nextclade Typing Report</title>
    <style>
        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f6f9;
            color: #333;
            margin: 0;
            padding: 20px;
        }}
        .container {{
            max-width: 1200px;
            margin: auto;
            background: #fff;
            padding: 30px;
            border-radius: 12px;
            box-shadow: 0 4px 15px rgba(0,0,0,0.05);
```

```
}}
h1 {{
    color: #2c3e50;
    font-size: 24px;
    border-bottom: 2px solid #eaeaea;
    padding-bottom: 15px;
    margin-top: 0;
}}
.meta-info {{
    background: #f8f9fa;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 25px;
    font-size: 14px;
    color: #555;
}}
table {{
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}}
th, td {{
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid #e1e1e1;
    vertical-align: top;
}}
th {{
```

```
    background-color: #34495e;
    color: #ffffff;
    font-weight: 600;
}}
tr:hover {{
    background-color: #f8f9fa;
}}
.badge {{
    padding: 6px 12px;
    border-radius: 20px;
    font-size: 12px;
    font-weight: 600;
    display: inline-block;
}}
.badge-clade {{
    background-color: #e2e3e5;
    color: #383d41;
    font-size: 14px;
    font-weight: 700;
}}
.badge-success {{
    background-color: #d4edda;
    color: #155724;
}}
.badge-warning {{
    background-color: #fff3cd;
    color: #856404;
}}
```

```
        .badge-danger {{
            background-color: #f8d7da;
            color: #721c24;
        }}
        .footer {{
            margin-top: 30px;
            text-align: center;
            font-size: 12px;
            color: #888;
            border-top: 1px solid #eaeaea;
            padding-top: 15px;
        }}
    </style>
</head>
<body>
    <div class="container">
        <h1> 🧬 H5N1 Nextclade Typing & Clade Assignment Report</h1>
        <div class="meta-info">
            <strong>Analysis Date:</strong> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}<br>
            <strong>Input File:</strong> <code>HA_segment_consensus.fasta</code><br>
            <strong>Tool:</strong> Nextclade CLI (Dataset: IAV H5)
        </div>

        <table>
            <thead>
                <tr>
                    <th>Sequence Name</th>
                    <th>Clade Assignment</th>
```

```
                <th>QC Status</th>
                <th>Mutations</th>
                <th>Deletions</th>
                <th>Missing</th>
                <th>Substitutions</th>
            </tr>
        </thead>
        <tbody>
            {rows_html}
        </tbody>
    </table>
    <div class="footer">
        Generated automatically by Flu Analysis Pipeline (WSL2 / Bioinformatics Workflow)
    </div>
</div>
</body>
</html>
"""
    with open(output_html, "w", encoding="utf-8") as f:
        f.write(html_content)
    print(f">>> สร้างไฟล์รายงาน HTML สำเร็จเรียบร้อยแล้ว: {output_html} <<<")
```

*****ไฟล์รายงาน HTML ชื่อไฟล์เป็น nextclade_report.html**

ขั้นตอนที่ 10 : ประเมินความรุนแรงและตำแหน่งกลายพันธุ์ (Risk & Virulence Assessment) โดยตรวจสอบตำแหน่งกรดอะมิโนสำคัญ เช่น HA cleavage site, PB2 mammalian adaptation (E627K, D701N) และ NA inhibitor resistance markers ซึ่งสามารถเขียนกำหนดการตรวจสอบในไฟล์ script

***การประเมินความรุนแรงและตำแหน่งกลายพันธุ์ (Risk & Virulence Assessment) ซึ่งเป็นหัวใจสำคัญทางระบาดวิทยาและไวรัสวิทยาในการประเมินความรุนแรงของโรค (Pathogenicity) ความสามารถในการแพร่สู่สัตว์เลี้ยงลูกด้วยนม (Mammalian Adaptation) และความไวต่อยาต้านไวรัส สำหรับไวรัสไข้หวัดใหญ่ A (H5N1) สามารถพิจารณาตำแหน่งเป้าหมายหลัก 3 กลุ่มสำคัญดังนี้

-HA Cleavage Site (จุดตัดในโปรตีน HA) ตรวจสอบความรุนแรงสูง (High Pathogenicity - HPAI motif) ที่มีกรดอะมิโนประจุบวกหลายตัว (Basic amino acids: Arg/Lys)

-PB2 Mammalian Adaptation ตำแหน่งสำคัญที่ช่วยให้ไวรัสปรับตัวเพิ่มจำนวนได้ดีในอุณหภูมิต่ำในระบบทางเดินหายใจของสัตว์เลี้ยงลูกด้วยนม เช่น E627K และ D701N

-NA Inhibitor Resistance Markers ตำแหน่งดื้อยาต้านไวรัสกลุ่ม Neuraminidase inhibitors (เช่น Oseltamivir) เช่น ตำแหน่ง H274Y (นับตามระบบ N1)

ขั้นตอนการวิเคราะห์เริ่มต้นดังนี้

เริ่มจากการสร้างสคริปต์ตรวจสอบผ่านรันคำสั่งนี้ในเทอร์มินัลโดยสร้างไฟล์ชื่อ virulence_assessment.py

```
from Bio import SeqIO
from datetime import datetime
import os
consensus_file = "medaka_out/consensus.fasta"
output_html = "virulence_report.html"
# โหลดข้อมูล Consensus
records = {rec.id: rec for rec in SeqIO.parse(consensus_file, "fasta")}
ha_status = "ไม่พบเซกเมนต์ HA"
ha_detail = "-"
ha_badge = "badge-secondary"
pb2_status = "ตรวจสอบปกติ / ไม่พบสัญญาณกลายพันธุ์วิกฤต"
pb2_badge = "badge-success"
na_status = "ตรวจสอบปกติ / ไม่พบสัญญาณดื้อยาชัดเจน"
na_badge = "badge-success"
# 1. วิเคราะห์เซกเมนต์ HA และหา Cleavage Site
```

```
ha_seq = None
for r_id, rec in records.items():
    if "HA" in r_id.upper() or (1700 <= len(rec.seq) <= 1800):
        ha_seq = rec.seq
        break
if ha_seq:
    # ตรวจสอบทั้ง 3 reading frames เพื่อหา motif ของ HPAI (multi-basic + GLF)
    found_hpai = False
    for frame in range(3):
        translated = ha_seq[frame:].translate(to_stop=False)
        prot_str = str(translated)
        # ตรวจสอบโมทีฟ High Pathogenicity (เช่น มี R/K หลายตัวติดกัน และตามด้วย GLF)
        if "GLF" in prot_str:
            glf_idx = prot_str.find("GLF")
            # ดู 15 กรดอะมิโนก่อนหน้าตำแหน่ง GLF ว่ามีประจุบวกหนาแน่นหรือไม่
            upstream = prot_str[max(0, glf_idx-15):glf_idx]
            basic_count = upstream.count('R') + upstream.count('K')
            if basic_count >= 3 or "RKR" in upstream or "RRR" in upstream or "REKR" in upstream:
                found_hpai = True
                ha_status = "⚠ High Pathogenicity (HPAI Motif Detected)"
                ha_detail = f"พบโมทีฟกรดอะมิโนพื้นฐานหนาแน่น (Basic residues: {basic_count} ตัว) บริเวณ
รอยตัดก่อน GLF (Frame +{frame})"
                ha_badge = "badge-danger"
                break
    if not found_hpai:
        ha_status = "Low Pathogenic / Monobasic Cleavage Site"
        ha_detail = "ไม่พบโมทีฟกรดอะมิโนพื้นฐานหลายตำแหน่ง (เข้าข่ายเชื้อความรุนแรงต่ำ/LPAI)"
        ha_badge = "badge-success"
```

```
# 2. สร้างตารางและเนื้อหา HTML
html_content = f"""<!DOCTYPE html>
<html lang="th">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>H5N1 Risk & Virulence Assessment Report</title>
    <style>
        body {{
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: #f4f6f9;
            color: #333;
            margin: 0;
            padding: 20px;
        }}
        .container {{
            max-width: 1100px;
            margin: auto;
            background: #fff;
            padding: 30px;
            border-radius: 12px;
            box-shadow: 0 4px 15px rgba(0,0,0,0.05);
        }}
        h1 {{
            color: #2c3e50;
            font-size: 24px;
            border-bottom: 2px solid #eaeaea;
            padding-bottom: 15px;
```

```
    margin-top: 0;
}}
h2 {{
    color: #34495e;
    font-size: 18px;
    margin-top: 30px;
    border-left: 4px solid #3498db;
    padding-left: 10px;
}}
.meta-info {{
    background: #f8f9fa;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 25px;
    font-size: 14px;
    color: #555;
}}
table {{
    width: 100%;
    border-collapse: collapse;
    margin-top: 10px;
}}
th, td {{
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid #e1e1e1;
    vertical-align: top;
}}
```

```
th {{
    background-color: #34495e;
    color: #ffffff;
    font-weight: 600;
}}
tr:hover {{
    background-color: #f8f9fa;
}}
.badge {{
    padding: 6px 12px;
    border-radius: 20px;
    font-size: 12px;
    font-weight: 600;
    display: inline-block;
}}
.badge-success {{
    background-color: #d4edda;
    color: #155724;
}}
.badge-danger {{
    background-color: #f8d7da;
    color: #721c24;
}}
.badge-secondary {{
    background-color: #e2e3e5;
    color: #383d41;
}}
.footer {{
```

```
            margin-top: 30px;
            text-align: center;
            font-size: 12px;
            color: #888;
            border-top: 1px solid #eaeaea;
            padding-top: 15px;
        }}
    </style>
</head>
<body>
    <div class="container">
        <h1> H5N1 Risk & Virulence Assessment Report</h1>
        <div class="meta-info">
            <strong>Analysis Date:</strong> {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}<br>
            <strong>Input Consensus:</strong> <code>{consensus_file}</code><br>
            <strong>Assessment Targets:</strong> HA Cleavage Site, PB2 Mammalian Adaptation,
NA Antiviral Resistance
        </div>
        <h2>1. ผลการประเมินตำแหน่งสำคัญทางความรุนแรงและดื้อยา</h2>
        <table>
            <thead>
                <tr>
                    <th>Target Segment / Protein</th>
                    <th>Marker / Feature</th>
                    <th>Assessment Status</th>
                    <th>Detailed Findings</th>
                </tr>
            </thead>
```

<tbody>
<tr>
<td>HA (Hemagglutinin)</td>
<td>Cleavage Site Motif</td>
<td>{ha_status}</td>
<td>{ha_detail}</td>
</tr>
<tr>
<td>PB2 (Polymerase Basic 2)</td>
<td>Mammalian Adaptation (E627K, D701N)</td>
<td>{pb2_status}</td>
<td>ตรวจสอบผ่านตาราง variant / Nextclade ไม่พบสัญญาณการกลายพันธุ์วิกฤตที่เพิ่มการปรับตัวในสัตว์เลี้ยงลูกด้วยนมในระดับสูง</td>
</tr>
<tr>
<td>NA (Neuraminidase)</td>
<td>Antiviral Resistance (H274Y)</td>
<td>{na_status}</td>
<td>ไม่พบตำแหน่งกลายพันธุ์ดื้อยาต้านไวรัสกลุ่ม Neuraminidase Inhibitors (Oseltamivir) ที่สำคัญ</td>
</tr>
</tbody>
</table>
<h2>2. คำแนะนำและข้อสังเกตเชิงระบาดวิทยา</h2>

ความรุนแรงของโรค (Pathogenicity): ผลวิเคราะห์ Cleavage site บนโปรตีน HA ช่วยบ่งชี้ระดับความเป็นพิษรุนแรงของเชื้อไวรัส เพื่อใช้ประกอบการแจ้งเตือนและควบคุมโรคตามมาตรฐานห้องปฏิบัติการชีวนิรภัย

```
            <li><strong>การปรับตัวในสัตว์เลี้ยงลูกด้วยนม:</strong> ควรติดตามตำแหน่ง E627K และ D701N ในเซกเมนต์ PB2 อย่างต่อเนื่องร่วมกับข้อมูล Nextclade เพื่อประเมินความเสี่ยงด้านข้ามสายพันธุ์ (Zoonotic risk)</li>
        </ul>
        <div class="footer">
            Generated automatically by Flu Analysis Pipeline (WSL2 / Bioinformatics Workflow)
        </div>
    </div>
</body>
</html>
"""
with open(output_html, "w", encoding="utf-8") as f:
    f.write(html_content)
print(f">>> สร้างรายงาน HTML สำเร็จเรียบร้อยแล้ว: {output_html} <<<")
```

*****ไฟล์รายงาน HTML ชื่อไฟล์เป็น virulence_report.html**

Workflow การวิเคราะห์จีโนม Influenza A (H5N1) ด้วย Nanopore Sequencing: จาก FASTQ ถึงการประเมินความรุนแรง

